

**ISUTC INSTITUTO SUPERIOR DE
TRANSPORTES E COMUNICAÇÕES**
PROGRAMAÇÃO I



Tema: **Métodos (funções)**

Sumário

- Conceito,
- Importância,
- Classificação de métodos em Java,
- Estrutura de um método e sua assinatura,
- Métodos com parâmetros,
- Parâmetros versus argumentos,
- Blocos e escopos [visibilidade de variáveis dentro e fora de métodos]

Tema: **Métodos (funções)**

Objectivos

- Conhecer o conceito de métodos/subprogramas;
- Identificar melhores locais para os utilizar;
- Conhecer os tipos de métodos existentes em java e convenções;
- Dominar a estrutura de um método em java;
- Construir o seu próprio método;
- Identificar onde declarar os diferentes tipos de variáveis;
- Diferenciar parâmetros e argumentos

Métodos (funções): definição

Um **método** representa um **bloco de código** que permite **separar** distintas partes dum programa em **pequenas e simples porções**, que podem ser invocados (chamados), uma ou várias vezes, se sua funcionalidade for necessária em um trecho da própria classe ou em outra classe.

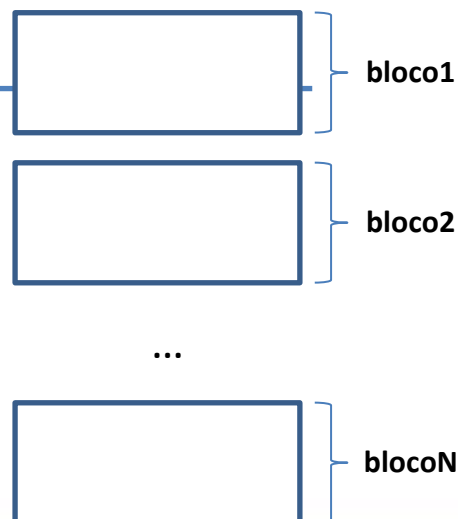
- Os principais motivos que levam a utilização de métodos se referem a redução do código de um sistema, à melhoria de modularização do sistema e a facilitação da manutenção do sistema.

Métodos (funções): classificação

Os métodos classificam-se pelo seu grau de importância em:

1) Organização

- Separa/divide operações complexas em blocos de código por forma a garantir maior performance do programa;
- Divide uma corrente de instruções de código em pequenas e específicas porções;

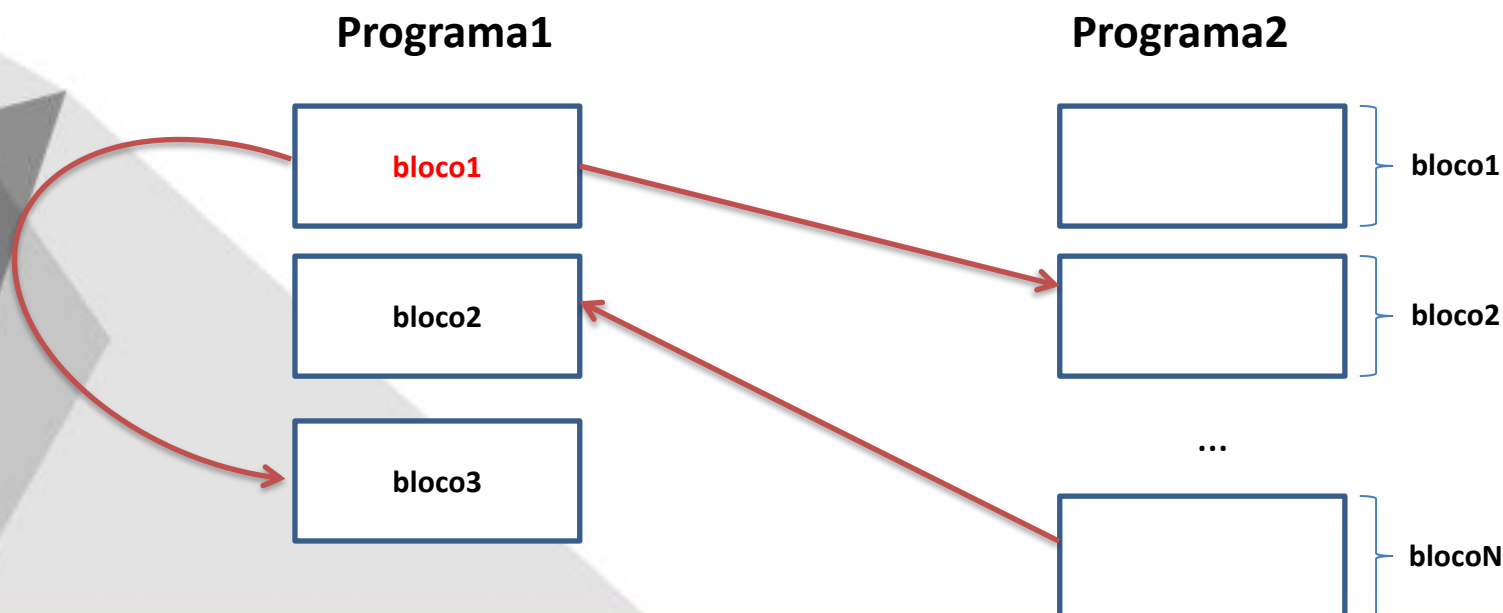


Métodos (funções): classificação

Os métodos classificam-se pelo seu grau de importância em:

2) Reusabilidade:

- Reduz a repetição de código;
- Maximiza o uso e aproveitamento do código



Métodos (funções): Estrutura de um método:

```
[acesso] [modificador] [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}
```

Métodos (funções): Estrutura de um método- acesso

- **public**: o método é visível por qualquer classe. É o qualificador mais aberto no sentido de que qualquer classe pode usar esse método.
- **private**: o método é visível apenas pela própria classe. É o qualificador mais restritivo.
- **protected**: o método é visível pela própria classe, por suas subclasses e pelas classes do mesmo pacote.

```
public [modificador] [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}  
  
private [modificador] [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}  
  
protected [modificador] [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}
```


Métodos (funções): Estrutura de um método- modificador

- **static:** o método é compartilhado por todos os objetos instanciados a partir da mesma classe. Um método static não pode acessar qualquer variável declarada dentro de uma classe (salvo se a variável estiver declarada também como **static**,
- **abstract,**
- **final,**
- **native,**
- **synchronized.**

```
[-] public static [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}  
  
[-] public abstract [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}  
  
[-] public final [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}  
  
[-] public native [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}  
  
[-] public synchronized [tipo_de_retorno] nomeMetodo([lista argumentos]){  
    <intrucoes>;  
    [com retorno ou sem retorno];  
}
```

Métodos (funções): Estrutura de um método - tipo de retorno

Tipo de retorno refere-se ao tipo de dado retornado pelo método

- ***void type*** : método que quando declarado, não devolvem um valor específico.
- ***return type*** : métodos que quando declarados, devolvem sempre um valor [referente a um determinado tipo de dado em java].

Estes por norma, terminam sempre com um *statement return*.

```
public static void nomeMetodo([lista argumentos]){  
    int x = 3, y = 5;  
    int soma = x + y;  
    System.out.println("Soma = "+soma);  
}
```

```
public static int nomeMetodo([lista argumentos]){  
    int x = 3, y = 5;  
    int soma = x + y;  
    return soma;  
}
```

```
public static String nomeMetodo([lista argumentos]){  
    String disciplina = "Programação 1";  
    return disciplina;  
}
```

```
public static int[] nomeMetodo([lista argumentos]){  
    int [] V = {1,15,-7,10,5};  
    return V;  
}
```

Métodos (funções): Estrutura de um método - nomeMetodo

Nome do método: utilizam as mesmas regras para dar nome uma variável, o que mais deve considerar:

- pode ser qualquer palavra ou frase, desde que iniciada por uma letra.
- Se o nome for uma frase, não pode conter espaços em branco.
- Por padrão, todo nome de método inicia com letra minúscula.
- O nome é constituído por verbos: **calcular, somar, imprimir**
- **Não pode dar declarar 2 métodos com a mesma estrutura, isto é, mesmo nome, tipo de retorno e parâmetros.**

```
public static void somar([lista argumentos]){  
    int x = 3, y = 5;  
    int soma = x + y;  
    System.out.println("Soma = "+soma);  
}  
  
public static int somarDoisValores([lista argumentos]){  
    int x = 3, y = 5;  
    int soma = x + y;  
    return soma;  
}  
  
public static String imprimirDisciplina([lista argumentos]){  
    String disciplina = "Programação 1";  
    return disciplina;  
}  
  
public static int[] retornarVetor([lista argumentos]){  
    int [] V = {1,15,-7,10,5};  
    return V;  
}
```

```
public class Teste {  
    ...  
    public void comer(String s) {  
        ...  
    }  
    public void comer(int i) {  
        ...  
    }  
    public void comer(double f) {  
        ...  
    }  
    public void comer(int i, double f) {  
        ...  
    }  
}
```

Apesar do mesmo nome de método e mesmo tipo todos tem tipos de parâmetros diferentes.

Métodos (funções): Estrutura de um método – Lista de argumentos

A lista de argumentos é a lista de valores que o método vai precisar, obedecendo a sintaxe:

[tipo1][nome1], [tipo2][nome],..., [tipoN][nomeN],

```
public static void somar() {  
    int x = 3, y = 5;  
    int soma = x + y;  
    System.out.println("Soma = "+soma);  
}  
  
public static int somarDoisValores(int x, int y) {  
    int soma = x + y;  
    return soma;  
}  
  
public static void imprimirVetor(int [] V) {  
    System.out.println(Arrays.toString(V));  
}
```


Métodos (funções): Parâmetros Vs Argumentos

Parâmetros, é um conjunto de nomes no formato
[tipo_de_dado] [nome_variável]

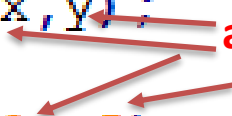
```
public static int soma(parâmetros int x, int y){  
    //calcula da soma de x e y  
    int soma=x+y;  
    return soma;  
}
```

Métodos (funções): Parâmetros Vs Argumentos

Argumentos são variáveis ou valores utilizados no chamamento de um método, com o objectivo de passar valores aos parâmetros definidos no método.

`variavel = nome_do_metodo([argumentos]);`

```
int x = 3;
int y = 5
int soma = soma(x, y);
// OU
int soma = soma(3, 5);
```



Métodos (funções): Parâmetros Vs Argumentos

- **Parâmetros:** aparecem na definição de um método, entre parênteses após o nome do método. Estes especificam que informação deverá ser passada ao método no momento da execução. Alguma literatura trata-os por **parâmetros formais**.
- **Argumentos:** valor que é passado a um método quando é executado e o seu valor é referenciado pelo nome do parâmetro durante a execução do método. Alguma literatura trata-os por **parâmetros reais**.

Métodos (funções): exemplo

```
public class Exemplo{  
    public static void somarDoisValores(int x, int y){  
        int soma = x + y;  
        System.out.println("soma = "+soma);  
    }  
  
    public static void main(String args []){  
        int x = 3, y = 5;  
        somarDoisValores(x,y);  
    }  
}
```

Métodos (funções): exemplo

```
import java.util.Scanner;

public class Exemplo{

    public static int somarDoisValores(int x, int y){
        int soma = x + y;
        return soma;
    }

    public static void calcularMedia(int x, int y){
        int soma = somarDoisValores(x,y);
        float media = (float) soma / 2;
        System.out.println("Media = "+media);
    }

    public static void main(String args []){
        Scanner ler = new Scanner(System.in);
        System.out.print("x = ");
        int x = ler.nextInt();
        System.out.print("y = ");
        int y = ler.nextInt();
        calcularMedia(x,y);
    }
}
```

Métodos (funções): variáveis membros

Existem vários tipos de variáveis:

- **Variáveis globais** em uma classe — são chamadas de campos (atributos).
- Variáveis em um método ou bloco de código — são chamadas de **variáveis locais**.
- Variáveis em declarações de métodos — são chamadas de **parâmetros**.

```
public class Teste{  
    // declaracao de variaveis globais  
    public static void main(String[] args){  
        // declaracao de variaveis locais  
        //instrucoes elementares e/ou chamadas de metodos  
    }  
  
    public static void nomeMetodo( /* declaracao de parametros*/ ){  
        // declaracao de variaveis locais  
        //instrucoes elementares e/ou chamadas de metodos  
    }  
}
```

```
public class NomeDaClasse{  
    /* a variável X abaixo é global */  
    static int x = 15;  
  
    public static void nomeDoMetodo ()  
    {  
        /* a variável Y abaixo é local; está definida somente dentro deste método */  
        int y = 14;  
        int soma = x + y;  
        for( int i = 0; i < 10; i++ ){  
            /* a variável i é local, definida só dentro deste bloco */  
            ... //  
        }  
    }  
}
```


Atenção:

- Uma variável declarada dentro de um método é considerada local ao método onde ela foi declarada;
- A variável local é criada sempre que o método onde ela foi declarada é executado e destruída quando ele termina a sua execução;
- É erro tentar aceder ou obter o valor de uma variável fora do método onde esta foi declarada;
- É possível declarar variáveis com mesmo nome em métodos diferentes pois ainda que tenham mesmo nome são diferentes e localizados em espaços de memória separados e com visibilidade diferente.

Atenção:

As variáveis globais são compostas por 3 componentes:

- Zero ou mais modificadores, como **public** ou **private**
- O tipo do campo(atributo)
- O nome do campo(atributo)

```
public class Teste {  
    // declaração de variaveis globais  
    public static int x;  
    private static int y;  
  
    public static void main(String args []) {
```

```
public class Teste {  
    public static int valor;  
  
    public static void main(String args []) {  
        valor = 5;  
        System.out.println("valor = "+valor);  
        mudarValor();  
        System.out.println("valor = "+valor);  
    }  
    public static void mudarValor() {  
        valor = 10;  
    }  
}
```

Imprime 5

Imprime 10

Exemplo:

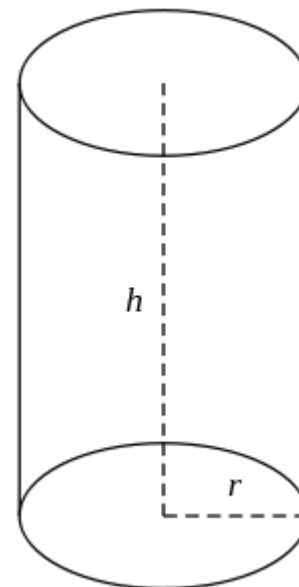
Elabore um programa que determina o volume de um cilindro, utilizando métodos.

$$V = A_b \times h$$

Onde:

$A_b \rightarrow$ Área da base;

$A_b \rightarrow$ Altura



GARANTE O TEU FUTURO
•————•
COM UMA FORMAÇÃO SÓLIDA



Prolong. da Av. Kim Il Sung (IFT/TDM) Edifício
D1
Maputo, Moçambique

www.facebook.com/isutc

www.transcom.co.mz/isutc